S2S-05: Step 5 — Execute: Configuration Import and Agent Cutover

> **Series:** S2S — SaaS to SaaS Migration | **Notebook:** 5 of 9 | **Phase:** Upgrade | **Step:** Execute | **Created:** March 2026 | **Last Updated:** 10/13/2026

Overview

With the target tenant provisioned and all prerequisites staged (Step 4), this step performs the actual migration: importing configuration, redirecting agents, and validating data flow. Execution follows a wave-based approach — non-production environments first, production last — with DQL validation gates between each wave.

This is the highest-risk step in the migration. Every action in this notebook changes the live monitoring path. Follow the configuration deployment order from Step 3 exactly, and validate after each major operation.

S2S Migration Framework

Phase	Steps	Focus
Plan	1. Discover → 2. Strategize → 3. Design	Understand what exists, choose approach, design target
Upgrade	4. Prepare → **5. Execute** → 6. Integrate	Build target, move config, connect integrations
Run	7. Expand → 8. Enable → 9. Optimize	Roll out agents, enable teams, tune and decommission

> **You are here: Step 5 — Execute.** The target tenant is provisioned, configuration is exported, ActiveGates are deployed, and the configuration freeze is active (Step 4).

> Now you import configuration and redirect agents.

Table of Contents

1. Configuration Deployment Order
2. Monaco Deploy Workflow
3. Terraform Apply for IAM
4. Entity ID Remapping
5. OneAgent Reconfiguration
6. Kubernetes Operator Migration
7. ActiveGate Migration
8. Wave Execution and Validation
9. Step Completion Checklist

Prerequisites

Requirement	Details
Step 4 Complete	Target tenant provisioned, SSO/IAM deployed, Monaco export validated, ActiveGates connected, K8s operator prepared
Configuration Freeze Active	Source tenant frozen — no new settings changes
Change Window Approved	Execution window approved by change management
Rollback Procedure Reviewed	Rollback steps documented and tested
Target Tenant Access	API token with `WriteConfig`, `settings.write`, `entities.read` scopes
Monaco CLI	`monaco v2.x` pointed at target tenant

Requirement	Details
Terraform	`terraform >= 1.0` with Dynatrace provider `~> 1.91`
Communication Plan	Stakeholders notified of execution window

Order of Operations

This notebook covers **operations 4–6** of the Upgrade phase:

#	Operation	Section	Dependency
1	Provision target tenant and configure SSO/IAM	Step 4	Step 3 design deliverables
2	Export configuration from source tenant	Step 4	Source tenant access
3	Deploy ActiveGates and prepare K8s operators	Step 4	Target tenant provisioned
4	**Import configuration to target tenant**	Sections 1–4	Operations 1–3 complete
5	**Redirect agents to target tenant**	Sections 5–7	Configuration imported
6	**Validate data flow in target tenant**	Section 8	Agents redirected
7	Reconnect cloud integrations	Step 6	Agents reporting
8	Migrate dashboards, workflows, and extensions	Step 6	Integrations connected

> **Bold** operations are covered in this notebook. Operations 1–3 were completed in Step 4. Operations 7–8 are covered in Step 6.

1. Configuration Deployment Order

Configuration must be imported in dependency order. Resources that other resources reference must exist first. This is the 24-item deployment order from Step 3 — follow it exactly.

Phase 1: Data Foundation — Deploy First

Order	Category	Tool	Validation
1	Grail buckets	Monaco / Terraform	Bucket list matches design
2	Enrichment rules (K8s labels → tags)	Monaco	Tags populating on incoming data
3	OpenPipeline processing rules	Monaco	Rules processing test data
4	Segments	Monaco / Terraform	Segments visible in UI

Phase 2: Gen2 (Classic) Configuration

Order	Category	Tool	Validation
5	Management zones	Monaco	Zones visible in UI
6	Auto-tags	Monaco	Tags applying to entities
7	Request attributes	Monaco	Attributes capturing on services
8	Calculated service metrics	Monaco	Metrics generating
9	Custom services and detection rules	Monaco	Services detected
10	Application detection rules	Monaco	Web applications created
11	Conditional naming rules	Monaco	Entity names updating
12	Anomaly detection settings	Monaco	Thresholds set
13	Credential vault entries	Manual	Secrets stored

Phase 3: Monitoring and Alerting

Order	Category	Tool	Validation
14	Alerting profiles	Monaco	Profiles visible
15	Notification rules + integrations	Monaco	Test notification sent

Order	Category	Tool	Validation
16	Maintenance windows	Monaco	Windows scheduled
17	SLO definitions	Monaco	SLOs evaluating (after agents report)
18	Synthetic monitors	Monaco	Monitors executing

Phase 4: User-Facing Configuration

Order	Category	Tool	Validation
19	Classic dashboards	Monaco	Dashboards load (data after agents)
20	Gen3 dashboards and notebooks	Monaco	Documents visible
21	Workflows and automations	Terraform / Monaco	Workflows listed

Phase 5: Governance — Deploy Last

Order	Category	Tool	Validation
22	IAM policies (tightened)	Terraform	Policies restrict as designed
23	IAM groups (final)	Terraform	Users inherit correct permissions
24	IAM policy bindings (final)	Terraform	Access validated by test users

> **Important:** Deploy Phases 1–4 **before** redirecting any agents. Agents will begin generating data immediately upon connection — the data foundation, processing rules, and alerting configuration must be in place first.

2. Monaco Deploy Workflow

Monaco deploys configuration in two stages: an offline dry-run, then the deploy. `monaco deploy` takes the manifest as its one positional argument and selects the target with `--`

`environment`; the target's URL and token come from the manifest, not from command-line flags.

Point the Manifest at the Target

Add the target environment to `export/manifest.yaml`. Both values name environment variables — the manifest never holds the token itself:

```
environmentGroups:
  - name: target
    environments:
      - name: target-tenant
        url:
          type: environment
          value: DT_TARGET_URL
        auth:
          token:
            name: DT_TARGET_TOKEN
```

```
export DT_TARGET_URL="https://&lt;target-env-id&gt;.live.dynatrace.com"
export DT_TARGET_TOKEN="dt0c01.xxx..."
```

Platform configurations (documents, automations, buckets, segments, SLOs, OpenPipeline) additionally need an OAuth client under the environment's `auth` block.

Dry-Run (Offline)

```
monaco deploy ./export/manifest.yaml --environment target-tenant --dry-run
```

A dry-run checks manifest structure, references and template rendering. It does **not** contact the target tenant, so it cannot tell you whether the target will accept the payloads — Monaco's own flag help: *"Dry-run will resolve all configuration parameters and render JSON templates, but can not validate the content of JSON payloads."*

Errors and Where They Surface

Error	Surfaces at	Cause	Fix
`reference not found`	Dry-run	A configuration references one that is not in the deploy	Include the referenced project, or deploy it first (see Section 1)
Template or parameter error	Dry-run	Unresolved parameter or unset environment variable	Fix the YAML or export the variable
`unknown schema` (HTTP 400)	Deploy	Schema or payload not valid on the target	Remove or adapt the setting for the target
`permission denied` (HTTP 403)	Deploy	Token missing a scope	Add the scope to the target token
`duplicate key`	Deploy	The object already exists on the target with a different ID	Download it from the target and reconcile, or delete it first

Deploy

```
# Deploy configuration to the target tenant
monaco deploy ./export/manifest.yaml --environment target-tenant

# Deploy a single project (phased deployment). monaco download writes one
project,
# named "project" by default — split configurations into separate project
folders
# to deploy them in phases.
monaco deploy ./export/manifest.yaml --environment target-tenant --project
project
```

> **Sources:** Monaco CLI commands (DT docs) — *"A dry-run doesn't connect to Dynatrace and can't validate the content of the JSON sent to Dynatrace."*, monaco deploy flags, v2.30.0 (Dynatrace GitHub).

> **Monaco resolves dependencies automatically** within a single deploy run. The 24-item deployment order (Section 1) is the reference model for understanding dependencies. If you deploy everything in one run, Monaco handles the ordering.

3. Terraform Apply for IAM

IAM policies with tightened `WHERE` clauses can now be applied because all referenced resources (buckets, schemas, segments) exist in the target tenant.

```
# Navigate to IAM Terraform directory
cd terraform/iam

# Plan with tightened policies
terraform plan -var-file="target-tenant.tfvars" -out=iam-tighten.plan

# Review the plan carefully — verify WHERE clauses reference existing
resources
terraform show iam-tighten.plan

# Apply tightened IAM
terraform apply iam-tighten.plan
```

IAM Tightening Example

```
# Before (Step 4 — broad access during setup)
resource "dynatrace_iam_policy" "sre_access" {
  name            = "SRE Team Access"
  environment     = var.dynatrace_environment_id
  statement_query = "ALLOW storage:logs:read, storage:spans:read,
storage:metrics:read; ALLOW storage:buckets:read;"
}

# After (Step 5 — scoped to specific buckets)
resource "dynatrace_iam_policy" "sre_access" {
  name            = "SRE Team Access"
  environment     = var.dynatrace_environment_id
  statement_query = &lt;&lt;-EOT
    ALLOW storage:logs:read
      WHERE storage:bucket-name IN ("app_logs", "infra_logs");
    ALLOW storage:spans:read
      WHERE storage:bucket-name = "default_spans";
    ALLOW storage:metrics:read;
    ALLOW storage:buckets:read
      WHERE storage:bucket-name IN ("app_logs", "infra_logs",
"default_spans");
    ALLOW storage:buckets:read
      WHERE storage:table-name = "metrics";
  EOT
}
```

> **Bucket permission.** Each table read is paired with `storage:buckets:read`, which the IAM policy reference (DT docs) describes as *"Required additionally to a table permission"*. Scoping that permission by `storage:bucket-name` is what limits a team to specific buckets.
>
> **Deploy IAM last** (Phase 5 in the deployment order). If you tighten IAM too early, it may block subsequent Monaco deploys or agent registrations.

4. Entity ID Remapping

Entity IDs are auto-generated and unique per tenant. Configurations exported from the source tenant contain hardcoded entity IDs that will not exist in the target tenant.

Remapping Strategy

Before Import	After Import
`entityId("HOST-1A2B3C4D")`	`tag("env:production"), tag("app:checkout")`
`type(SERVICE),entityId(SERVICE-5E6F7A8B)`	`type(SERVICE),tag("service:checkout")`
`dt.entity.host == "HOST-1A2B3C4D"` (DQL on logs, spans, metrics)	Filter on a dimension the data already carries: `fetch logs | filter host.name == "prod-web-01"` , or `timeseries avg(dt.host.cpu.usage), filter:{host.name == "prod-web-01"}`

Automated Remapping Script

```
# Find all hardcoded entity IDs in the exported configuration
grep -rn "HOST-\|SERVICE-\|PROCESS_GROUP-\|APPLICATION-" ./export/project/ \
  | grep -v ".git" \
  &gt; entity-id-references.txt

# Count references per entity type
echo "Hosts: $(grep -c 'HOST-' entity-id-references.txt)"
echo "Services: $(grep -c 'SERVICE-' entity-id-references.txt)"
echo "Process Groups: $(grep -c 'PROCESS_GROUP-' entity-id-references.txt)"
echo "Applications: $(grep -c 'APPLICATION-' entity-id-references.txt)"
```

The DQL row is dimension-first: `host.name` is a dimension on the host metrics and on log records, so the filter needs no entity lookup and survives the move (both forms executed on the validation tenant, 09/28/2026). Host names can collide across an estate — check uniqueness before relying on a name-only match (**FAQ-25** §4).

> **Best practice:** Replace hardcoded entity IDs with entity selectors or tags in the exported configuration **before** deploying to the target tenant. This makes the configuration portable and resilient to future migrations.

5. OneAgent Reconfiguration

OneAgent instances must be redirected from the source tenant to the target tenant. This is done host-by-host (or wave-by-wave) using the `oneagentctl` command.

Reconfiguration Command

```
# On each monitored host, run as root/administrator:

# Linux
/opt/dynatrace/oneagent/agent/tools/oneagentctl \
  --set-server=https://&lt;target-env-id&gt;.live.dynatrace.com/communication
\
  --set-tenant=&lt;target-env-id&gt; \
  --set-tenant-token=&lt;target-tenant-token&gt; \
  --restart-service

# Windows
"C:\Program Files\dynatrace\oneagent\agent\tools\oneagentctl.exe" \
  --set-server=https://&lt;target-env-id&gt;.live.dynatrace.com/communication
\
  --set-tenant=&lt;target-env-id&gt; \
  --set-tenant-token=&lt;target-tenant-token&gt; \
  --restart-service
```

> **OneAgent and application restarts required.** `--set-tenant` and `--set-tenant-token` always go together — the environment ID changes with the tenant. The OneAgent CLI docs: *"These parameters require restart of OneAgent, as well as restart of all the applications monitored with deep code modules. Add --restart-service to the command to restart OneAgent automatically (version 1.189+) or stop and start OneAgent process manually."* `--restart-service` handles OneAgent; plan the application restarts as part of the wave execution.
>
> **Sources:** OneAgent configuration via command-line interface (DT docs).

ActiveGate Routing for Firewall-Restricted Hosts

On-premises or DMZ hosts that cannot reach the SaaS target directly must route through ActiveGates. Use the ActiveGate communication endpoint instead of the SaaS URL:

```
# Route through a single ActiveGate
/opt/dynatrace/oneagent/agent/tools/oneagentctl \
  --set-server=https://&lt;activegate-host&gt;:9999/communication \
  --set-tenant=&lt;target-env-id&gt; \
  --set-tenant-token=&lt;target-tenant-token&gt; \
  --restart-service

# Route through multiple ActiveGates (failover — semicolon-separated)
/opt/dynatrace/oneagent/agent/tools/oneagentctl \
  --set-server="https://ag-01.corp.com:9999/communication;https://ag-
02.corp.com:9999/communication" \
  --set-tenant=&lt;target-env-id&gt; \
  --set-tenant-token=&lt;target-tenant-token&gt; \
  --restart-service
```

> **Network dependency:** ActiveGate routing requires firewall rules allowing the monitored host to reach the AG on port 9999. Coordinate with the network team early — firewall change lead times are a common critical path item.

Wave Strategy

Wave	Scope	Validation Window	Rollback Window
Wave 1	Non-production (dev, staging)	2–4 hours	1 hour
Wave 2	Low-risk production (internal apps)	4–8 hours	2 hours
Wave 3	High-risk production (customer-facing)	24 hours	4 hours
Wave 4	Remaining infrastructure	24 hours	4 hours

Automation at Scale

For environments with hundreds of hosts, use configuration management tools:

Tool	Approach
Ansible	Playbook with `oneagentctl` commands, host groups matching waves
Puppet/Chef	Update OneAgent configuration module with target tenant URL
SCCM/Intune	Push script to Windows fleet
SSM Run Command	Execute on AWS EC2 instances by tag

> **Tip — OneAgent Attribute Enrichment (1.333+):** During agent redirect, consider adding primary tags and fields in the same call using `--set-host-tag` (for example `--set-host-tag="dt.cost.costcenter=12345"`). This enriches all telemetry at the source with `primary_tags.environment`, `dt.security_context`, `dt.cost.costcenter`, etc. — eliminating the need for some server-side auto-tagging rules. See docs.

Validate Host Migration

After each wave, verify hosts are reporting to the target tenant:

```
// Target tenant: count hosts reporting after wave cutover (Smartscape)
smartscapeNodes "HOST", from:-2h
| summarize host_count = count()
| fieldsAdd validation = "Compare this count against the pre-migration
baseline from Step 4"

// Use the same surface as the Step 4 baseline (Smartscape vs classic).
Classic fallback:
// fetch dt.entity.host | summarize host_count = count()
```

```
// Target tenant: count services detected after agent cutover (Smartscape)
smartscapeNodes "SERVICE", from:-2h
| summarize service_count = count()
| fieldsAdd validation = "Compare against the Step 4 baseline once the wave's
traffic has flowed"

// Classic fallback: fetch dt.entity.service | summarize service_count =
count()
```

6. Kubernetes Operator Migration

A DynaKube cannot be edited to point at a different tenant: `spec.apiUrl` is immutable, and the Operator rejects a CR that changes the tenant in it. Kubernetes environments are migrated by **deleting** the DynaKube and **applying a new one** against the target tenant (the same rule as **M2S-05**).

Migration Steps

Step	Command	Notes
1	Confirm the source CR is saved for rollback	`dynakube-source.yaml` from Step 4, Section 5
2	Delete the running DynaKube	`kubectl delete dynakube dynakube -n dynatrace`
3	Apply the target CR prepared in Step 4	Its `tokens:` field names the `dynakube-target` secret created in Step 4
4	Monitor operator logs for successful reconnection	`kubectl logs -n dynatrace -l app.kubernetes.io/name=dynatrace-operator`
5	Verify pods restart with new configuration, then restart applications	`kubectl get pods -n dynatrace -w`

Delete and Recreate the DynaKube

```
# apiUrl is immutable — delete and recreate (OneAgent pods restart)
kubectl delete dynakube dynakube -n dynatrace

# Apply the target CR prepared in Step 4 (tokens: dynakube-target)
kubectl apply -f dynakube-target.yaml -n dynatrace

# Monitor the rollout
kubectl get dynakube -n dynatrace -w

# Verify all OneAgent pods are running with new configuration
kubectl get pods -n dynatrace -l app.kubernetes.io/component=oneagent
```

> **Monitoring gap.** Deleting the DynaKube removes its OneAgent and ActiveGate pods, so monitoring on that cluster stops until the new CR reconciles. Restart the monitored applications afterwards so their code modules report to the target. Schedule the switch inside the wave's maintenance window.

The Dynatrace docs give the same sequence: *"Delete the existing DynaKube (starting with Dynatrace Operator version 1.3.0, editing spec.apiUrl is not allowed)."* Their example reuses the secret name `dynakube`; this series creates a separate `dynakube-target` secret in Step 4, so the source tokens stay available for rollback.

Helm-installed Operators are switched the same way: the tenant URL and tokens live in the DynaKube CR and its secret, so use the delete-and-recreate steps above rather than looking for a Helm value to change.

> **Sources:** Migrate Dynatrace Operator to a new environment (DT docs).
>
> **Rollout timing:** the new DynaKube's OneAgent pods start on every node, and rollout time grows with cluster size. The docs give no figure, so time the first non-production cluster and plan the maintenance window from that measurement. Watch progress with `kubectl get pods -n dynatrace -w`.

7. ActiveGate Migration

Source tenant ActiveGates are decommissioned after target tenant ActiveGates are validated and all agents are redirected.

Migration Sequence

Step	Action	Validation
1	Verify target AGs are healthy and processing data	AG appears in target tenant topology
2	Verify no agents are still routing through source AGs	Source AG connection count = 0

Step	Action	Validation
3	Disable source AG services	Stop AG process, do not uninstall yet
4	Monitor for 24–48 hours	Verify no data gaps in target tenant
5	Uninstall source AG	Remove AG software and clean up

> **Do not uninstall source ActiveGates until Step 9 (Optimize).** Keeping them stopped but installed allows rapid rollback if needed.

8. Wave Execution and Validation

After each wave of agent cutover, run these DQL queries against the **target tenant** to validate data flow.

Validation Query Suite

Run all queries below after each wave. All should return data within 15 minutes of agent cutover.

```
// Target tenant: verify log data is flowing
fetch logs, from:-15m
| summarize log_count = count()
| fieldsAdd status = if(log_count &gt; 0, then: "Logs flowing", else: "No logs
— check agent connectivity")
```

```
// Target tenant: verify span/trace data is flowing
fetch spans, from:-15m
| summarize span_count = count()
| fieldsAdd status = if(span_count &gt; 0, then: "Spans flowing", else: "No
spans — check OneAgent instrumentation")
```

```
// Target tenant: verify metric data is flowing (CPU usage as smoke test)
timeseries avg_cpu = avg(dt.host.cpu.usage), from:-15m
| fieldsAdd avg_cpu_value = arrayAvg(avg_cpu)
| fieldsAdd status = if(avg_cpu_value &gt; 0, then: "Metrics flowing", else:
"No metrics — check host agent")
```

```
// Target tenant: verify enrichment tags are applied to incoming data
fetch logs, from:-15m
| filter isNotNull(dt.security_context)
| summarize enriched = count()
| fieldsAdd status = if(enriched &gt; 0, then: "Enrichment active", else: "No
enriched records — check OpenPipeline rules")
```

```
// Target tenant: check for detected problems generated during migration
//
// Corrected 08/12/2026: the cell sorted by `timestamp` AFTER a `fields`
projection that dropped it,
// so it failed with FIELD_DOES_NOT_EXIST. dt.davis.problems does carry
`timestamp` — `fields` had
// simply removed it. Keep the sort key in the projection.
fetch dt.davis.problems, from:-1h
| fields timestamp, display_id, event.name, event.status, event.category
| sort timestamp desc
| limit 20
```

Validation Decision Matrix

Validation	Result	Action
Host count ≥ 95% of baseline	Pass	Proceed to next wave
Host count 80–95% of baseline	Warning	Investigate missing hosts before next wave
Host count < 80% of baseline	Fail	**Halt and investigate** — do not proceed
Log count > 0 in target	Pass	Log pipeline functional
Span count > 0 in target	Pass	Distributed tracing functional
Metric data present	Pass	Infrastructure monitoring functional
detected problems from migration	Expected	Review each — suppress if migration-related

Validation	Result	Action
Enrichment tags present	Pass	OpenPipeline rules functional
Enrichment tags missing	Fail	Fix OpenPipeline rules before next wave

9. Step Completion Checklist

Before proceeding to Step 6 (Integrate), verify all execution tasks are complete:

Deliverable	Status	Owner	Notes
Configuration imported (Phases 1–4)	☐	Platform	Monaco deploy completed without errors
IAM policies tightened (Phase 5)	☐	Security / Platform	Terraform apply successful
Entity ID remapping completed	☐	Platform	No hardcoded IDs in deployed config
Wave 1: Non-prod agents redirected	☐	Platform / App teams	Data flowing, validation passed
Wave 2: Low-risk prod agents redirected	☐	Platform / App teams	Data flowing, validation passed
Wave 3: High-risk prod agents redirected	☐	Platform / App teams	Data flowing, 24h validation passed
Wave 4: Remaining agents redirected	☐	Platform / Infra	All agents reporting to target
K8s operators migrated	☐	Platform / K8s	All clusters reporting
Source ActiveGates stopped	☐	Infra	AG processes stopped (not uninstalled)
detected problems reviewed	☐	Platform	Migration-related problems documented
Host count matches baseline	☐	Platform	Within 5% of pre-migration count

> **Gate check:** Do not proceed to Step 6 until all agents are reporting to the target tenant and validation queries return expected results. Cloud integrations (Step 6) depend on agents providing entity context.

Next Step

Continue to **S2S-06: Step 6 — Integrate: Cloud, Dashboards, and Workflows** to reconnect cloud integrations, migrate dashboards, and restore all external connections.

Completed	Next	Remaining
~~1. Discover~~ → ~~2. Strategize~~ → ~~3. Design~~ → ~~4. Prepare~~ → ~~5. Execute~~	**6. Integrate**	7. Expand → 8. Enable → 9. Optimize

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.